🎬 Shockwave — demo recording guide (≤ 3:00)

Read the **bold VO** lines aloud; do the `[ON SCREEN]` actions. Target ~2:50.

Pre-flight (do this once before recording)

```
# 1. Orbit graph is indexed (Flask is the demo repo)
orbit list                      # should show demo-repos\flask · indexed

# 2. Start the web app
pip install -e ".[web]"
shockwave-web                   # → http://127.0.0.1:7777   (leave running)
```

Open these browser tabs, in this order: 1. `http://127.0.0.1:7777/` — the landing page 2. `http://127.0.0.1:7777/app` — the Blast Monitor 3. MR with the live bot comment: `https://gitlab.com/uthmannabeel-group/Shockwave-project/-/merge_requests/1` 4. The agent: `https://gitlab.com/explore/ai-catalog/agents/1011457/`

Recording setup: 1080p, browser zoom ~110%, hide bookmarks bar, dark desktop. A terminal window ready for the one CLI moment. Tools: OBS / Loom / Xbox Game Bar (`Win+G`).

The script

0:00 – 0:15 · Hook

`[ON SCREEN]` Landing page (tab 1), slowly.

> **"You're reviewing a one-line change. What does it break? `grep` can't tell you — it can't follow a cross-file call graph. But GitLab Orbit already built one. This is Shockwave."**

0:15 – 1:10 · The Blast Monitor (the hero)

`[ON SCREEN]` Click **Launch Blast Monitor** → type `setupmethod` → **Detonate**.

> **"I'm changing `setupmethod` — a tiny internal decorator in Flask. Watch the blast radius ripple out: 114 definitions across 10 files depend on it."**

`[ON SCREEN]` Point at the **HIGH RISK 100/100** badge, then the cyan nodes.

> **"Shockwave gives a verdict — HIGH risk — and shows it's reachable from a public entry point: Flask's `@route` decorator. That call path means a break here is externally observable, not internal."**

`[ON SCREEN]` Scroll the right rail to **Tests to run**.

> **"And instead of running the whole suite, it tells me the 58 tests that actually exercise this change — copy, paste, run. Plus a generated test stub for the hotspots nothing covers."**

1:10 – 1:35 · It's the real graph, in the cloud

`[ON SCREEN]` Terminal:

```
shockwave analyze compute --remote https://gitlab.com --token $env:GITLAB_TOKEN
```

> **"Same engine, no local index — this queries the hosted Orbit graph live over its REST API. Real callers. Indexed facts, not guesses."**

(If your token has expired, skip the live command and say this over the committed `examples/` output — the next two beats already prove live Orbit use.)

1:35 – 2:15 · The autonomous reviewer (the payoff)

`[ON SCREEN]` Switch to the **MR tab (3)**; scroll to Shockwave's comment.

> **"Now put it in the pipeline. Open a merge request, and Shockwave's bot posts this review automatically — the risk verdict, the untested hotspots, the exact tests to run — straight from Orbit. Intelligent orchestration, with context."**

2:15 – 2:45 · Five surfaces

`[ON SCREEN]` Briefly: the **agent page (tab 4)**, then back to the landing's "five surfaces".

> **"It's a CLI, this web monitor, the merge-request bot, a GitLab Duo agent in the AI Catalog, and a reusable Orbit skill — one graph, everywhere review happens."**

2:45 – 3:00 · Close

`[ON SCREEN]` Landing hero.

> **"Shockwave: see the blast radius, the exposure, and the tests you're missing — before you click merge. Review with context, not guesswork."**

After recording

- Upload to **YouTube or Vimeo** (unlisted is fine), keep it **≤ 3:00**.
- Title: *"Shockwave — know what a change breaks, before you merge (GitLab Orbit)"*.
- Paste the link into Devpost (write-up ready in `DEVPOST.md`).

Fallbacks (if anything misbehaves live)

- App won't load → hard-refresh; confirm `shockwave-web` is running on 7777.
- Remote token expired → use the committed reports in `examples/flask/` ; the MR-bot comment is already live and public.
- Everything is reproducible offline from `examples/` — you can narrate over those if needed.